

Type Coercion & Equality

Section 2 — JavaScript Fundamentals

JavaScript-এর সবচেয়ে interesting এবং একই সাথে সবচেয়ে confusing অংশগুলোর একটি হলো **Type Coercion & Equality**।

অনেক সময় JavaScript একটি value-এর type automatically পরিবর্তন করে ফেলে। আবার `==` এবং `===` একই-
looking comparison হলেও তাদের behavior একেবারে আলাদা।

এই chapter-এ আমরা শিখব:

1. **Type Coercion — Implicit vs Explicit**
2. `==` vs `===` — **Loose vs Strict Equality**
3. `NaN`, `Infinity`, `-Infinity`

1. What Is Type Coercion?

Type coercion হলো JavaScript-এর এমন একটি behavior যেখানে একটি value-কে অন্য data type-এ convert করা হয়।

সহজভাবে:

```
এক type-এর value
    ↓
অন্য type-এ conversion
    ↓
operation চালানো
```

Example:

```
const result = "5" + 2;

console.log(result);
```

Output:

```
"52"
```

এখানে:

```
"5" → string  
2 → number
```

কিন্তু result:

```
"52" → string
```

JavaScript এখানে number `2`-কে string-এর সাথে কাজ করার জন্য string-এর মতো treat করেছে।

এটাই type coercion-এর একটি example।

2. Type Conversion vs Type Coercion

দুইটি term আলাদা:

Type Conversion

Developer নিজে type পরিবর্তন করে।

```
const value = "123";  
  
const number = Number(value);
```

এখানে আমরা explicitly বলেছি:

`"123"`-কে number বানাও।

এটা **explicit conversion**।

Type Coercion

JavaScript নিজে conversion করে।

```
const result = "123" - 1;
```

Output:

```
122
```

এখানে আমরা `Number("123")` লিখিনি।

JavaScript নিজেই string-কে number-এ convert করেছে।

এটা **implicit coercion**।

3. Explicit vs Implicit Conversion

দুইটা term খুব ভালোভাবে মনে রাখো:

Type	কে conversion করছে?
Explicit conversion	Developer
Implicit coercion	JavaScript

4. Explicit Type Conversion

JavaScript-এ type manually convert করার জন্য common functions:

```
String()
Number()
Boolean()
```

5. String Conversion

```
const value = 123;

const result = String(value);

console.log(result);
console.log(typeof result);
```

Output:

```
123
string
```

Number `123` → String `"123"` হয়েছে।

5.1 More Examples

```
String(100);  
  // "100"  
  
String(true);  
  // "true"  
  
String(false);  
  // "false"  
  
String(null);  
  // "null"  
  
String(undefined);  
  // "undefined"
```

6. Number Conversion

String থেকে number:

```
const value = "123";  
  
const result = Number(value);  
  
console.log(result);  
console.log(typeof result);
```

Output:

```
123  
number
```

6.1 More Examples

```
Number("10");  
// 10  
  
Number("10.5");  
// 10.5  
  
Number("0");  
// 0  
  
Number("");  
// 0  
  
Number("hello");  
// NaN  
  
Number(true);  
// 1  
  
Number(false);  
// 0  
  
Number(null);  
// 0  
  
Number(undefined);  
// NaN
```

শেষের কয়েকটি conversion বিশেষভাবে মনে রাখার মতো।

7. Boolean Conversion

`Boolean()` কোনো value-কে `true` অথবা `false` বানায়।

```
Boolean(1);  
    // true  
  
Boolean(0);  
    // false  
  
Boolean("hello");  
    // true  
  
Boolean("");  
    // false
```

এখান থেকেই আসে JavaScript-এর **Truthy & Falsy** concept।

এটা পরের related chapter-এ আরও বিস্তারিতভাবে আসবে।

8. Unary Plus Operator

Number conversion-এর একটি short form হলো unary `+`।

```
const value = "100";  
  
console.log(+value);
```

Output:

```
100
```

কারণ:

```
+"100"
```

প্রায় equivalent:

```
Number("100")
```

8.1 Examples

```
+ "10";  
// 10  
  
+ "10.5";  
// 10.5  
  
+ "hello";  
// NaN  
  
+ true;  
// 1  
  
+ false;  
// 0  
  
+ null;  
// 0  
  
+ undefined;  
// NaN
```

9. Implicit Type Coercion

এখন আসল interesting অংশ।

JavaScript অনেক operation-এর সময় automatically type conversion করে।

Example:

```
console.log("5" - 2);
```

Output:

```
3
```

JavaScript `"5"` string-কে number-এ convert করেছে।

10. Operator-এর Special Behavior

 operator একটু special।
এটি দুইভাবে কাজ করতে পারে:

Number addition
অথবা
String concatenation

10.1 Number + Number

```
console.log(5 + 2);
```

Output:

```
7
```

10.2 String + String

```
console.log("5" + "2");
```

Output:

```
"52"
```

10.3 String + Number

```
console.log("5" + 2);
```

Output:

```
"52"
```

এখানে string concatenation হয়েছে।

10.4 Number + String

```
console.log(5 + "2");
```

Output:

```
"52"
```

11. Why Does "5" + 2 Become "52" ?

Mental model:

```
"5" + 2
  ↓
এক পাশে string আছে
  ↓
+ concatenation হিসেবে কাজ করে
  ↓
2 → "2"
  ↓
"5" + "2"
  ↓
"52"
```

তাই:

```
"5" + 2
```

কে:

```
7
```

ভাবলে ভুল হবে।

12. , , Coercion

, ,  সাধারণত numeric operation করে।

Example:

```
console.log("10" - 2);
```

Output:

```
8
```

```
console.log("10" * 2);
```

Output:

```
20
```

```
console.log("10" / 2);
```

Output:

```
5
```

JavaScript এখানে string  -কে number-এ coerce করেছে।

13. Some Classic Coercion Examples

এগুলো JavaScript interview-এ খুব common:

```
"5" + 2;  
// "52"  
  
"5" - 2;  
// 3  
  
"5" * 2;  
// 10  
  
"5" / 2;  
// 2.5
```

আর:

```
5 + true;  
// 6
```

কারণ:

```
true → 1
```

তাই:

```
5 + 1 = 6
```

14. Boolean Coercion

```
5 + true;
```

Output:

```
6
```

কারণ:

```
true → 1
```

আর:

```
5 + false;
```

Output:

```
5
```

কারণ:

```
false → 0
```

15. **null** Coercion

Numeric context-এ:

```
Number(null);
```

Output:

```
0
```

তাই:

```
5 + null;
```

Output:

```
5
```

16. **undefined** Coercion

Numeric conversion:

```
Number(undefined);
```

Output:

```
NaN
```

তাই:

```
5 + undefined;
```

Output:

```
NaN
```

17. Equality in JavaScript

দুইটি value equal কিনা check করার জন্য JavaScript-এ প্রধানত:

```
==  
===
```

দুইটি operator ব্যবহার করা হয়।

কিন্তু তাদের behavior আলাদা।

18. — Loose Equality

 comparison করার সময় প্রয়োজন হলে type coercion করতে পারে।

Example:

```
console.log(5 == "5");
```

Output:

```
true
```

কারণ JavaScript comparison-এর আগে type conversion করতে পারে।

Mental model:

```
5 == "5"

type different
  ↓
coercion
  ↓
5 == 5
  ↓
true
```

19. `===` — Strict Equality

`===` value এবং type—দুটোই compare করে।

```
console.log(5 === "5");
```

Output:

```
false
```

কারণ:

```
5    → number
"5"  → string
```

Type same নয়।

20. `==` vs `===`

Feature	<code>==</code>	<code>===</code>
Value compare	✓	✓
Type compare	Coercion করতে পারে	✓
Automatic conversion	সম্ভব	✗
Strict	✗	✓
Recommended	সাধারণত avoid	✓

Modern JavaScript-এ সাধারণত:

```
===
```

ব্যবহার করা best practice।

21. More Examples

```
5 == "5";  
// true  
  
5 === "5";  
// false
```

```
true == 1;  
// true  
  
true === 1;  
// false
```

```
false == 0;  
// true  
  
false === 0;  
// false
```

```
null == undefined;  
// true  
  
null === undefined;  
// false
```

এই শেষ example-টি বিশেষভাবে interesting।

22. Why Is `null == undefined` True?

Loose equality (`==`) JavaScript-এর special coercion rules follow করে।

```
null == undefined
```

এর result:

```
true
```

কিন্তু:

```
null === undefined
```

হবে:

```
false
```

কারণ strict equality-তে তাদের type/value identity এক নয়।

Practical code-এ এই ধরনের coercion-এর উপর depend না করে সাধারণত explicit checks বা `===` ব্যবহার করা ভালো।

23. Why Prefer `===` ?

ধরো:

```
const userInput = "0";
```

এখন:

```
if (userInput == 0) {  
  // ...  
}
```

এখানে unexpected coercion হতে পারে।

বরং:

```
if (userInput === 0) {  
    // ...  
}
```

ব্যবহার করলে string `"0"` number `0` হিসেবে accidentally equal হবে না।

এতে code-এর behavior predictable হয়।

24. Important Exception — `Object.is()`

JavaScript-এ equality check-এর আরেকটি method:

```
Object.is()
```

এটি `===`-এর সাথে অনেক ক্ষেত্রে একই behavior দেয়, কিন্তু কিছু special case আলাদা।

সবচেয়ে famous:

```
Object.is(NaN, NaN);  
// true
```

কিন্তু:

```
NaN === NaN;  
// false
```

আর:

```
Object.is(-0, 0);  
// false
```

যেখানে:

```
-0 === 0;  
// true
```

`Object.is()` advanced JavaScript-এ useful।

25. What Is NaN ?

NaN মানে:

Not-a-Number

নামটি confusing, কারণ NaN নিজে JavaScript-এর number type-এর একটি special numeric value।

```
console.log(typeof NaN);
```

Output:

```
"number"
```

26. How Do We Get NaN ?

Invalid numeric operation থেকে NaN পাওয়া যেতে পারে।

```
console.log(Number("hello"));
```

Output:

```
NaN
```

আর:

```
console.log(0 / 0);
```

Output:

```
NaN
```

আর:

```
console.log(Math.sqrt(-1));
```

Output:

NaN

27. NaN Is Not Equal to Itself

এটা JavaScript-এর famous behavior:

```
console.log(NaN === NaN);
```

Output:

```
false
```

এমনকি:

```
console.log(NaN == NaN);
```

Output:

```
false
```

তাই:

```
value === NaN
```

দিয়ে NaN check করা উচিত নয়।

28. Correct Way to Check NaN

ব্যবহার করো:

```
Number.isNaN(value)
```

Example:

```
const result = Number("hello");

console.log(Number.isNaN(result));
```

Output:

```
true
```

29. `Number.isNaN()` vs Global `isNaN()`

JavaScript-এ পুরোনো/global function:

```
isNaN()
```

এবং modern safer check:

```
Number.isNaN()
```

দুটো এক নয়।

Example:

```
isNaN("hello");
// true
```

কারণ global `isNaN()` আগে coercion করতে পারে।

কিন্তু:

```
Number.isNaN("hello");
// false
```

কারণ `"hello"` নিজে `NaN` নয়; এটি string।

Best practice:

```
Number.isNaN(value)
```

ব্যবহার করা।

30. Infinity

`Infinity` হলো JavaScript-এর একটি special numeric value যা positive infinity represent করে।

```
console.log(1 / 0);
```

Output:

```
Infinity
```

31. -Infinity

Negative infinity:

```
console.log(-1 / 0);
```

Output:

```
-Infinity
```

এটিও `number` type।

```
typeof Infinity;  
// "number"
```

32. Infinity-এর সাথে Operations

```
Infinity + 100;  
// Infinity
```

```
Infinity * 2;  
// Infinity
```

```
Infinity / 2;  
// Infinity
```

আর:

```
Infinity - Infinity;  
// NaN
```

কারণ infinity থেকে infinity subtract করার নির্দিষ্ট finite numeric result নেই।

33. Checking Infinity

একটি value finite কিনা check করতে:

```
Number.isFinite(value)
```

ব্যবহার করা যায়।

Example:

```
console.log(Number.isFinite(100));  
// true  
  
console.log(Number.isFinite(Infinity));  
// false  
  
console.log(Number.isFinite(NaN));  
// false
```

34. `Number.isFinite()` vs Global `isFinite()`

Global:

```
isFinite("100");  
// true
```

কারণ global `isFinite()` coercion করতে পারে।

কিন্তু:

```
Number.isFinite("100");  
// false
```

কারণ `"100"` string।

এখানেও:

```
Number.isFinite() বেশি predictable।
```

35. `Number.isInteger()`

কোনো value integer কিনা check করতে:

```
Number.isInteger(value)
```

Example:

```
Number.isInteger(10);  
// true  
  
Number.isInteger(10.5);  
// false  
  
Number.isInteger("10");  
// false
```

36. Safe Integers

JavaScript `number` type IEEE-754 double-precision floating-point format ব্যবহার করে।

এর কারণে সব integer unlimited precision-এ accurately represent করা যায় না।

Safe integer range:

```
Number.MIN_SAFE_INTEGER  
  
Number.MAX_SAFE_INTEGER
```

সাধারণভাবে:

```
-9,007,199,254,740,991  
to  
9,007,199,254,740,991
```

এর বাইরে exact integer arithmetic-এর প্রয়োজন হলে `BigInt` useful হতে পারে।

37. `Number.MAX_SAFE_INTEGER`

```
console.log(Number.MAX_SAFE_INTEGER);
```

Output:

```
9007199254740991
```

এর চেয়ে বড় integer-এর ক্ষেত্রে precision problem হতে পারে।

38. `Number.isSafeInteger()`

```
console.log(Number.isSafeInteger(100));
```

Output:

```
true
```

কিন্তু:

```
console.log(  
  Number.isSafeInteger(9007199254740992)  
);
```

Output:

```
false
```

39. A Practical Example — User Input

Browser form থেকে সাধারণত input value string হিসেবে আসে।

ধরো:

```
const age = "31";
```

তখন:

```
console.log(typeof age);
```

Output:

```
string
```

যদি arithmetic করতে হয়:

```
const numericAge = Number(age);  
  
console.log(numericAge + 1);
```

Output:

```
32
```

এটা explicit conversion-এর practical use।

40. A Common Real-World Bug

ধরো shopping cart:

```
const price = "500";  
const quantity = "2";  
  
const total = price + quantity;  
  
console.log(total);
```

Output:

"5002"

অনেকে আশা করবে:

1000

কিন্তু দুইটিই string হওয়ায়  concatenation করেছে।

Correct:

```
const total = Number(price) * Number(quantity);  
  
console.log(total);
```

Output:

1000

41. A Useful Mental Model for Coercion

JavaScript-এর coercion মনে রাখার সবচেয়ে useful way:



String থাকলে → concatenation হওয়ার সম্ভাবনা

 ,  , 

সাধারণত → numeric coercion



comparison-এর সময় coercion করতে পারে

===

type coercion করে না

তবে মনে রাখবে, JavaScript coercion rules অনেক বিস্তৃত। শুধু এই shortcut দিয়ে সব edge case predict করা যাবে না।

42. Best Practices

Rule 1 — Prefer ===

```
if (age === 18) {  
  }  
}
```

সাধারণত:

```
if (age == 18) {  
  }  
}
```

এর চেয়ে predictable।

Rule 2 — User Input Explicitly Convert করো

```
const age = Number(inputValue);
```

তারপর calculation করো।

Rule 3 — Number.isNaN() ব্যবহার করো



```
value === NaN;
```



```
Number.isNaN(value);
```

Rule 4 — `Number.isFinite()` ব্যবহার করো

numeric value expected হলে:

```
Number.isFinite(value);
```

দিয়ে validate করা useful।

Rule 5 — Implicit Coercion-এর উপর অতিরিক্ত depend করো না

এই ধরনের code:

```
const result = value + "";
```

কাজ করলেও explicit conversion বেশি readable:

```
const result = String(value);
```



Exercises

Exercise 1 — Explicit Conversion

নিচের value-গুলোকে number-এ convert করো:

```
"100"  
"25.5"  
"0"
```

তারপর তাদের type check করো।

Exercise 2 — Boolean Conversion

নিচের প্রতিটির result আগে predict করো:

```
Boolean(0)
Boolean(1)
Boolean("")
Boolean("hello")
Boolean(null)
Boolean(undefined)
```

তারপর browser console-এ verify করো।

Exercise 3 — Coercion

নিজে output predict করো:

```
console.log("10" + 5);
console.log("10" - 5);
console.log("10" * 5);
console.log("10" / 5);
```

Exercise 4 — Equality

Predict:

```
console.log(10 == "10");
console.log(10 === "10");

console.log(true == 1);
console.log(true === 1);

console.log(null == undefined);
console.log(null === undefined);
```

Exercise 5 — NaN

একটি variable তৈরি করো যার value `NaN` হবে।

তারপর:

1. `typeof` check করো।
2. `=== NaN` দিয়ে check করো।
3. `Number.isNaN()` দিয়ে check করো।

কোনটি কাজ করে এবং কেন?

Exercise 6 — Shopping Cart

ধরো API থেকে data এসেছে:

```
const price = "1500";  
const quantity = "3";
```

এমন code লেখো যাতে total হয়:

```
4500
```

এবং result-এর type হয়:

```
number
```



Output Prediction

এখানে **answer** না দেখে আগে নিজে **solve** করো।

Question 1

```
console.log("5" + 2);
```

Question 2

```
console.log("5" - 2);
```

Question 3

```
console.log("10" * "2");
```

Question 4

```
console.log(5 + true);
```

Question 5

```
console.log(5 + false);
```

Question 6

```
console.log(5 + null);
```

Question 7

```
console.log(5 + undefined);
```

Question 8

```
console.log(5 == "5");  
console.log(5 === "5");
```

Question 9

```
console.log(false == 0);  
    console.log(false === 0);
```

Question 10

```
console.log(null == undefined);  
    console.log(null === undefined);
```

Question 11

```
console.log(NaN === NaN);
```

Question 12

```
console.log(typeof NaN);
```

Question 13

```
console.log(1 / 0);  
    console.log(-1 / 0);
```

Question 14

```
console.log(Infinity - Infinity);
```

Question 15 — Challenge

```
console.log("10" + 5 - 2);
```

Step by step চিন্তা করে answer বের করো।

Interview Questions

1. Type coercion কী?
2. Type conversion এবং type coercion-এর পার্থক্য কী?
3. Implicit coercion কী?
4. Explicit conversion কী?
5. `"5" + 2` এর output `"52"` কেন?
6. `"5" - 2` এর output `3` কেন?
7. `==` এবং `===`-এর মধ্যে পার্থক্য কী?
8. কেন modern JavaScript-এ `===` prefer করা হয়?
9. `null == undefined` কেন `true`?
10. `null === undefined` কেন `false`?
11. `NaN` কী?
12. `typeof NaN` কী এবং কেন?
13. `NaN === NaN` কেন `false`?
14. `NaN` check করার correct way কী?
15. `isNaN()` এবং `Number.isNaN()`-এর পার্থক্য কী?
16. `Infinity` কী?
17. `-Infinity` কীভাবে তৈরি হতে পারে?
18. `Infinity - Infinity` কেন `NaN`?
19. `Number.isFinite()` কী?
20. `Number.isSafeInteger()` কখন ব্যবহার করা হয়?
21. JavaScript-এর safe integer range কত?
22. `BigInt` কেন দরকার হতে পারে?
23. `Object.is()` কী?
24. `Object.is(NaN, NaN)` এবং `NaN === NaN`-এর behavior আলাদা কেন?

Challenge 1 — Safe Number Converter

একটি function তৈরি করো:

```
toNumber(value)
```

যেটা string number-কে number-এ convert করবে।

Example:

```
toNumber("100");  
// 100  
  
toNumber("25.5");  
// 25.5
```

কিন্তু invalid input দিলে:

```
toNumber("hello");  
// "Invalid number"
```

Challenge 2 — Safe Division

একটি function তৈরি করো:

```
divide(a, b)
```

যেটা:

```
divide(10, 2);  
// 5
```

দেবে।

কিন্তু:

```
divide(10, 0);
```

এর ক্ষেত্রে meaningful error/message handle করবে।

Challenge 3 — Shopping Cart

এই data দেওয়া হলো:

```
const price = "1200";  
const quantity = "3";
```

একটি function তৈরি করো যেটা total price return করবে।

Expected:

```
3600
```

তোমাকে নিশ্চিত করতে হবে calculation-এর আগে data সঠিক type-এ আছে।



Mini Challenge — User Age Validator

ধরো user form থেকে age এসেছে:

```
const inputAge = "25";
```

একটি function তৈরি করো:

```
validateAge(inputAge)
```

Requirements:

- input number-এ convert করবে।
- invalid number detect করবে।
- age 18 বা তার বেশি হলে "Adult" return করবে।
- তার কম হলে "Minor" return করবে।
- NaN বা invalid input handle করবে।
- unnecessary == ব্যবহার করবে না।

Example:

```
validateAge("25");  
    // "Adult"  
  
validateAge("15");  
    // "Minor"  
  
validateAge("hello");  
    // "Invalid age"
```

Chapter Mental Model

পুরো chapter-টা এভাবে মনে রাখো:

```
TYPE COERCION  
|  
├─ Explicit  
|   ├── String()  
|   ├── Number()  
|   └─ Boolean()  
|  
└─ Implicit  
    ├── "5" + 2 → "52"  
    ├── "5" - 2 → 3  
    ├── true → 1  
    └─ false → 0
```

Equality:

```
==  
└─ Loose equality  
    └─ coercion করতে পারে  
  
===  
└─ Strict equality  
    └─ type + value compare
```

Special Numbers:

NaN

└ invalid numeric result

Infinity

└ positive infinity

-Infinity

└ negative infinity

Quick Revision

Type Conversion

```
String(value);  
Number(value);  
Boolean(value);
```

Loose Equality

```
5 == "5";  
// true
```

Strict Equality

```
5 === "5";  
// false
```

NaN

```
Number.isNaN(value);
```

Infinity

```
Number.isFinite(value);
```

Important

```
typeof NaN;
// "number"

typeof Infinity;
// "number"

NaN === NaN;
// false

null == undefined;
// true

null === undefined;
// false
```



Final Takeaway

JavaScript-এর coercion system প্রথমে confusing মনে হতে পারে। কিন্তু তিনটি core idea মনে রাখলে অনেকটা সহজ হয়ে যায়:

1. **JavaScript** কখনো কখনো নিজে **type convert** করে — এটিই **implicit coercion**!
2. **==** coercion করতে পারে, কিন্তু **===** type coercion না করে **strict comparison** করে।
3. **NaN**, **Infinity**, এবং **-Infinity** হলো **number** type-এর **special values**!

Modern JavaScript development-এর জন্য সবচেয়ে practical habit হলো:

```
const
let
===

Number()
String()
Boolean()

Number.isNaN()
Number.isFinite()
```

এর মাধ্যমে code-এর behavior যতটা সম্ভব **explicit, predictable** এবং **readable** রাখা।